DS314: GENERATIVE AI & LLM SYSTEMS · ACADEMIC LAB MODULE

DS314 - Topic 2: Introduction to Building with Large Language Models

Format: Jupyter Notebook (.ipynb) **Source:** DS314 - Topic 2 - Introduction to Building with Large Language Models.ipynb **Render Engine:** KaTeX + MathML (Zero-Delim Math) **Academic Level:** 3rd Year BS Data Science

DS314 — DS Elective 1 | Generative AI

Week 2 - Introduction to Building with Large Language Models

Intended Learning Outcome (ILO): Explain the importance of artificial intelligence and generative AI, identify major changes introduced by AI, and configure the required development environment.

Introduction to Building with LLMs

Overview: This learning portfolio introduces the basic idea behind building applications with Large Language Models (LLMs). Rather than training an AI model from scratch, developers can use existing foundation models and integrate their intelligence into applications through APIs.

In this lesson, you will explore the basic workflow of using an LLM in an application. You will connect to Google's Gemini API, send prompts to a pre-trained model, and observe how the model generates responses. You will then extend this basic chatbot by providing it with information from a PDF document, `jollibee-info.pdf`, allowing the chatbot to answer questions using additional context.

By the end of this learning portfolio, you should understand the basic flow of building with an LLM: using an existing model, connecting to it through an API, providing it with instructions and context, and generating responses through a simple chatbot.

Prerequisites: To follow along, you should have:

- Basic Python programming knowledge
- Familiarity with running Python notebooks (e.g., Jupyter or VS Code)
- An environment with Python 3.x and the ability to install libraries via `pip`
- A free **Gemini API key** from Google AI Studio

Section 1: LLM Applications — The New AI Revolution

LLMs are **Large Language Models**, a class of AI models trained on massive amounts of text data to understand and generate human-like language. These models (like Google's Gemini or OpenAI's GPT-4) can perform a range of language tasks: from answering questions and writing essays to translating languages and generating code. LLMs have become so powerful and versatile that building applications on top of them — LLM apps — is now a major trend in technology.

LLM Apps: The Key to the New AI

In recent years, generative AI (AI that can create content) has exploded in popularity and impact. A prime example is ChatGPT, a chatbot interface built by OpenAI on their GPT series model. ChatGPT's launch demonstrated the massive demand and potential for LLM-powered applications, it reached 100 million users just two months after launch, making it the fastest-growing consumer app in history. This unprecedented adoption underscored that LLM apps are key to the new AI era, bringing AI capabilities directly to millions of users.

LLM apps refer to any software or tool that leverages an underlying large language model to perform intelligent tasks. ChatGPT itself is an LLM app (a chat interface using an AI model), but there are many others:

- Writing assistants that help draft emails or articles
- Code helpers (like GitHub Copilot) that generate programming code
- Customer service bots that answer questions
- Personalized tutors or advisors in education apps

What makes LLM apps revolutionary is that you don't need to be an AI expert to create useful AI-driven tools. The model provides the intelligence (learned from vast data), and developers can focus on designing the application logic and user experience. In essence, LLMs are like a new kind of general-purpose engine for software, much like electricity or the internet, they can power many different applications with minimal effort. This is why we say LLM apps are "the key to the new AI." They allow AI to be plugged into all sorts of products and services.

From a societal perspective, these applications are driving significant changes. Creative work, customer support, content generation, and many other fields are being augmented or automated by LLM-based tools. Businesses are investing heavily in generative AI: in a 2023 survey, three in four business leaders (74%) identified generative AI as the top emerging technology that will impact their business in the near term. The ability to quickly build apps on top of powerful language models means AI solutions are being developed and deployed at an unprecedented pace.

LLM Apps and the Universalization of AI

The rise of LLM apps has led to the **universalization of AI**, meaning AI is becoming universally accessible and integrated into everyday tools. In the past, advanced AI might have been limited to research labs or big tech companies. Today, thanks to accessible APIs and frameworks, even individual developers or small startups can create AI applications that rival those from large companies.

Hosted, API-accessible models have especially accelerated this trend. A student, hobbyist, or small business no longer needs a research lab's budget or a personal GPU server to build a working AI application, they simply need an API key. This is exactly why this course's official model, **Google's Gemini**, is accessed through a free-tier API rather than a self-hosted, locally-run model: it keeps the barrier to entry at zero cost and minimal setup, which is the whole point of universalization in practice, not just in theory.

Real-world implications of this "AI for everyone" movement include:

- **Education:** Students and teachers use AI tutors or assistants.
- **Healthcare:** Doctors have AI summarize patient notes or research literature.
- **Business:** Teams use AI for writing reports, generating marketing copy, or analyzing data.
- **Personal use:** Individuals have AI-powered apps for improving writing, planning travel itineraries, or just having conversations.

Because LLM apps can be tailored to specific domains by feeding them relevant data or context, we see AI permeating niche areas too. For instance, a small business can deploy a custom chatbot to answer FAQs using its own documents as knowledge, without training a new model from scratch. This universal availability and adaptability of AI is what "universalization of AI" refers to, AI is no longer confined to tech giants; it's becoming a built-in feature across industries and communities.

ChatGPT vs. LLM Apps

It's important to distinguish ChatGPT (as an example of a single AI service) from the broader concept of LLM apps:

- **ChatGPT** is a specific application, a chat-based interface provided by OpenAI that lets users interact with their GPT models. It's a general-purpose assistant: you can ask it anything or have it generate any kind of text. However, ChatGPT is essentially a closed system (the data and model are hosted by OpenAI) and it serves a broad audience with general knowledge up to its training cutoff.
- **LLM Apps** is a broad term for any application built using an LLM. An LLM app could be a customized chatbot like ChatGPT, but it could also be an AI writing tool, a code generator, a translation system, etc. Crucially, when you build an LLM app, you have control over how the model is used: you can integrate it with specific data, enforce certain behavior, or embed it into a larger software system.

In simpler terms, think of ChatGPT as a powerful car you can rent, it's ready-to-go but you can't peek under the hood or modify the engine. Building your own LLM app is like constructing a custom vehicle using an engine (the LLM) and adding the features or accessories you need. You get to decide what data to load into it and how it interacts with users or other systems.

Why build an LLM app if ChatGPT is available? There are several reasons:

- **Customization:** With your own app, you can give the LLM custom knowledge or context. For example, feeding it company-specific documents so it can answer questions about your business (ChatGPT by default won't know your internal data).
- **Privacy:** Organizations might prefer not to send data to an external, general-purpose service. Building an LLM app gives you control over exactly what data goes where.
- **Specialization:** You can prompt (or, more advanced, fine-tune) the model to specialize in one task, whereas ChatGPT is a generalist.
- **Tool integration:** LLM apps can be combined with other software or tools. For instance, you might create a chatbot that not only answers from an LLM, but also fetches up-to-date information from a document or database. This kind of "augmented" application goes beyond what ChatGPT's base interface offers.

Analogy: ChatGPT vs LLM apps can be likened to using a public library vs. building a personal book collection. The library (ChatGPT) has a wide range of books (knowledge) and you just go and use it as-is. An LLM app is like curating your own collection: you get to include the books you need (specific data), arrange them how you want (program the interaction), and you can keep it private, but it requires effort to set up and maintain. Both have their place, and often they complement each other.

Key takeaway: ChatGPT is a single (very advanced) example of an LLM app provided as a service. By learning to build LLM apps, you unlock the ability to create your own AI assistants or tools tailored to different needs. In the next sections, we'll move from concept to practice: understanding the foundations of AI and generative AI, and then actually constructing a simple LLM-powered chatbot step by step, **using our course's official model, Gemini.**

Section 2: Generative AI Fundamentals and Environment Setup

Now that we've covered what LLM apps are and why they matter, let's step back and look at the bigger picture: Artificial Intelligence and Generative AI. This will give context to the technology behind LLMs. We'll also set up our programming environment using this course's official tools, then walk through building a mini chatbot application that leverages an LLM, using a real dataset.

2.1 Introduction to AI and Generative AI

Artificial Intelligence (AI) is a broad field of computer science that aims to create machines capable of performing tasks that typically require human intelligence. This includes problem-solving, understanding language, recognizing images, making decisions, and more. Traditional AI systems often rely on explicit rules or algorithms crafted by humans. In contrast, modern AI (especially machine learning and deep learning) uses data-driven approaches: the system "learns" from examples rather than being manually programmed for every scenario.

Within AI, **Generative AI** refers to systems that can generate new content. This could mean writing text, composing music, creating images, or even designing products. Generative models learn the patterns of the input data (like the structure of language or the style of images) and use that to produce original output that follows similar patterns. In the context of language:

- Older AI might answer questions by retrieving facts (think of a search engine or a FAQ system).
- A generative AI model like Gemini creates an answer in natural language, which may combine information from various sources or rephrase things in a new way.

LLMs are a type of generative AI specialized for text. They don't just recall memorized responses; they actually generate sentences word-by-word (technically, token-by-token) based on probabilities learned during training. This is why a model like Gemini can produce responses it has never seen before, it's formulating text on the fly, guided by patterns it learned from its training data.

Generative AI is important because it has introduced qualitative changes in how we interact with technology:

- We can have open-ended conversations with computers, blurring the line between human and machine communication.
- Content creation can be automated or assisted. For instance, copywriters use AI to get a draft that they then refine, speeding up the writing process.
- It lowers barriers to entry for many tasks. Someone who isn't a skilled artist can prompt an image AI to create artwork; a non-programmer can ask a code generator to write simple programs.

At the same time, generative AI introduces challenges:

- The generated content can sometimes be incorrect or nonsensical (AI "hallucinations").
- It can be used to produce misinformation or biased content if not properly checked.
- There are concerns about intellectual property (e.g., models trained on copyrighted text or images might output something too similar to the originals).

Despite these challenges, the momentum behind generative AI is huge. Businesses and communities are actively exploring how to harness it effectively. It's clear that generative AI (and LLMs in particular) are not just a tech fad but a fundamental shift in computing.

2.2 Changes Introduced by Modern AI

To appreciate the significance of LLM apps, let's highlight some key changes and trends introduced by modern AI:

- **Human-like Interaction:** Computers can now engage in conversations, understand context, and produce responses that feel naturally worded. This makes technology more accessible to people who can literally talk or write to their devices to get things done.
- **Automation of Creative Work:** AI is no longer just doing repetitive, formulaic tasks; it's also tackling creative endeavors. We see AI-generated articles, designs, music, and more. This doesn't replace human creativity but augments it, for example, helping a designer generate many concepts to choose from.
- **Personalization at Scale:** AI can tailor content for individual users. In education, an AI tutor can adapt to a student's learning pace. In marketing, AI can generate product descriptions focusing on different user demographics.
- **New Ethical and Social Questions:** With AI writing and producing content, questions arise about plagiarism, authenticity, and trust. These changes mean that anyone building or using AI apps must consider not just *can we do this*, but *should we*, and how to do it responsibly.
- **Job Transformation:** AI is shifting the skills demanded in many jobs. Routine writing or data analysis tasks might be automated, but this also creates demand for new skills, like AI prompt engineering (crafting good inputs for models), or oversight roles to monitor AI outputs. Rather than outright replacing humans, AI often acts as a powerful assistant, allowing people to focus on higher-level or more complex aspects of their work.

In summary, AI, especially generative AI, is changing how we work, communicate, and create. It's an exciting time where learning to work with AI is becoming an essential skill, much like computers or the internet in earlier eras.

To ground this knowledge, let's briefly introduce OpenAI (the organization behind ChatGPT) and then look at the ecosystem we'll actually be using in this course.

2.3 OpenAI Introduction

OpenAI is an AI research lab and company that has been at the forefront of generative AI. Founded in 2015, OpenAI's mission is to ensure artificial general intelligence (AGI) benefits all of humanity. OpenAI created the GPT series of models (GPT-2, GPT-3, GPT-3.5, GPT-4, and beyond), among the most advanced LLMs available. They also created other generative models like DALL-E (for images) and Codex (for code).

Their approach was initially non-profit and open research, but as the models grew more powerful and expensive to train, OpenAI transitioned to a "capped-profit" model and partnered with companies like Microsoft for funding and infrastructure. For end-users and developers, OpenAI provides:

- **ChatGPT:** the public-facing chat website (using GPT models).
- **OpenAI API:** a cloud API where developers send requests to the model and get the model's response back. The API is paid, with a certain cost per token of input/output.

It's worth knowing OpenAI's API exists and how it's typically used, since you'll encounter it constantly in industry. Conceptually, calling it looks like this:

```
# Example only — NOT what we use in this course, shown for awareness.
import openai
openai.api_key = "YOUR_API_KEY"
```

```
response = openai.chat.completions.create(
    model="gpt-4",
    messages=[{"role": "user", "content": "Explain the importance of AI in 3 sentences."}]
)
print(response.choices[0].message.content)
```

OpenAI's models are top-of-class in quality. However, they are proprietary and require paying for usage from the very first request. **This course uses a different, cost-free path instead: Google's Gemini API**, covered next.

2.4 This Course's Ecosystem: Google Gemini

Rather than relying on OpenAI's paid API or self-hosting an open-weight model on local hardware, **this course's official model is Google's Gemini**, accessed through the Gemini API's free tier. This choice is entirely practical, not a judgment on model quality:

- **No cost barrier:** every student can obtain a free API key from Google AI Studio and make real API calls throughout the semester, no billing setup required.
- **No hardware requirement:** unlike self-hosting an open-weight model (which typically needs a GPU with several gigabytes of memory), Gemini runs entirely on Google's servers, your laptop just sends the request and receives the response.
- **Actively maintained and current:** Gemini's model lineup (e.g. Gemini Flash for speed and cost-efficiency, Gemini Pro for higher capability) is updated regularly by Google.

You should still know that other ecosystems exist and work differently:

- **Anthropic's Claude** family is accessed similarly to OpenAI's GPT, through a paid API, with Claude.ai as the first-party chat application.
- **Open-weight models** (e.g. Meta's LLaMA family, Mistral, Falcon) can be downloaded and run on your own hardware instead of accessed through an API. This avoids per-request cost but requires a capable GPU and more setup, which is why we don't use this path in a classroom setting where hardware access varies widely between students.

For the rest of this course, when you see "the model" or "the LLM" referenced in code, it refers to Gemini, accessed through Google's official Python SDK.

2.5 Environment Setup for LLM App Development (in VS Code, step by step)

This course's official development setup uses **VS Code** for building real projects, so let's set everything up properly, step by step, rather than assuming it's already done.

Step 1: Install VS Code

Download and install VS Code from the official site if you haven't already. Once installed, open it.

Step 2: Install the required VS Code extensions

Inside VS Code, open the Extensions panel (the icon that looks like four squares on the left sidebar) and install:

- **Python** (by Microsoft) — gives VS Code the ability to run and debug Python code
- **Jupyter** (by Microsoft) — lets VS Code open and run `.ipynb` notebook files like this one directly, with the same cell-by-cell workflow you'd get in a browser-based notebook

Step 3: Create a project folder

Create a new folder for this course (e.g. `DS314`), and open that folder in VS Code (`File → Open Folder...`). Keep all of this week's files inside it.

Step 4: Create a virtual environment

A virtual environment keeps this course's Python packages separate from anything else installed on your machine, which avoids version conflicts later. Open VS Code's integrated terminal (`Terminal → New Terminal`, or `Ctrl+`` `) and run:

```
python -m venv venv
```

This creates a `venv` folder inside your project. Activate it:

- **Windows:** `venv\Scripts\activate`
- **Mac/Linux:** `source venv/bin/activate`

You'll know it worked if your terminal prompt now shows `(venv)` at the start of the line.

Step 5: Install the official course libraries

With your virtual environment active, install everything this notebook needs:

```
pip install google-genai langchain langchain-google-genai langchain-community langchain-text-splitters langchain-chroma chromadb pypdf python-dotenv
```

- `google-genai` — Google's current, officially supported SDK for calling Gemini models directly. (Note: an older package named `google-generativeai` also exists and may appear in older tutorials, it has been deprecated in favor of `google-genai`, so we install the current one from the start.)
- `langchain`, `langchain-google-genai`, `langchain-community`, `langchain-text-splitters`, `langchain-chroma` — LangChain is shown throughout this course as one convenient way to structure LLM applications (chains, retrievers, memory), not the only way. These packages connect LangChain to Gemini (`langchain-google-genai`, which itself now runs on the current `google-genai` SDK internally), to document-loading utilities (`langchain-community`), to text-splitting utilities (`langchain-text-splitters`), and to our vector store (`langchain-chroma`).
- `chromadb` — our official vector store's underlying engine, used for similarity search over document chunks.
- `pypdf` — for reading PDF files (we have a PDF dataset, `jollibee-info.pdf`).
- `python-dotenv` — lets us keep our API key in a separate, private file instead of typing it directly into our code (see Step 7).

Step 6: Select the right Python interpreter/kernel in VS Code

Open this notebook (`.ipynb` file) in VS Code. In the top-right corner of the notebook, click **Select Kernel**, and choose the Python interpreter inside your `venv` folder (it should be listed, often labeled with the folder name). This tells VS Code to run your code using the virtual environment you just set up, with all the packages you just installed, instead of your computer's default, separate Python installation.

Step 7: Store your API key safely using a `.env` file

How to Get a Free Gemini API Key

7.1. **Go to Google AI Studio.** Open https://aistudio.google.com/ in your browser and sign in with any Google account.

7.2. **Open the API key page.** Click **Get API key** (usually in the left sidebar or top-right of the page).

7.3. **Create a new key.** Click **Create API key**. You'll be asked to either:

- Create it under a new Google Cloud project (Google will set this up automatically for you), or
- Select an existing project if you already have one. For this course, creating a new project is fine, you don't need to configure anything else.

7.4. **Copy your key.** Once generated, your API key will appear as a long string starting with `AIza...`. Click the copy icon next to it.

7.5. **Never paste it directly into a notebook cell you plan to submit or share.** Instead, in your project folder, create a new file named exactly `.env` (yes, starting with a dot, no filename before it) and add one line:

`GEMINI_API_KEY=your_actual_key_goes_here`

7.6. **Confirm the free tier is active.** New keys are free-tier by default, no credit card required. You can check your usage and limits anytime from the same AI Studio page under **Usage**.

7.7. **Keep it private.** If you ever accidentally expose your key (e.g., commit it to GitHub), go back to the API key page and delete/regenerate it immediately, treat it like a password.

Save it. This file should **never** be submitted or shared, if you're using version control, add `.env` to a `.gitignore` file so it never gets uploaded by accident.

Step 8: Verify your setup

Run the code cell below. If it prints your key's first few characters without errors, your environment is correctly set up.

In [20]:
```
# Run this cell to confirm your environment is set up correctly.
# This does NOT make an API call yet, it just checks that your .env file is being read.

from dotenv import load_dotenv
import os

load_dotenv()  # reads the .env file in your project folder

api_key = os.getenv("GEMINI_API_KEY")

if api_key:
    print(f"API key loaded successfully. Starts with: {api_key[:4]}...")
else:
    print("No API key found. Check that your .env file exists, is named exactly '.env',")
    print("and is in the same folder as this notebook.")
```

Out [20]:
```
API key loaded successfully. Starts with: AIza...
```

Checkpoint: If you saw "No API key found," what are the two most likely causes?

Answer: Either (1) the `.env` file doesn't exist yet or isn't named exactly `.env` (watch out for `.env.txt`, a common mistake if your operating system hides file extensions), or (2) the `.env` file isn't in the same folder as this notebook, `load_dotenv()` looks in the current working directory by default.

2.6 Demo: Using an LLM to Generate Text (Hello, AI!)

Before diving into building a full application, let's do a quick demo to get comfortable with using an LLM in Python. We'll use Google's current `google-genai` SDK to generate text, a very basic LLM app that completes a prompt.

Example: We'll prompt the model to explain a concept in a few sentences.

In [21]:
```
# Requires: a free Gemini API key from Google AI Studio, stored in your .env file (Step 7 above)
# This cell needs internet access to actually run.

from google import genai
from dotenv import load_dotenv
import os

load_dotenv()

client = genai.Client(api_key=os.getenv("GEMINI_API_KEY"))

prompt = "Explain the importance of AI in 3 sentences."
response = client.models.generate_content(
    model="gemini-3.6-flash",
    contents=prompt
)

print(response.text)
```

Out [21]:
```
Artificial intelligence is critical because it automates complex tasks and processes vast amounts of data far beyond human speed and capacity. By driving innovation in vital fields like healthcare, finance, and environmental science, AI helps solve some of humanity's most pressing challenges. Ultimately, it enhances human capability and serves as the foundational technology shaping the future of work, society, and global progress.
```

Line-by-line explanation:

- `load_dotenv()` reads your `.env` file so the key becomes available to `os.getenv(...)`.
- `genai.Client(api_key=...)` creates a client object, this is the current, official way to authenticate with Gemini's API. (An older package, `google-generativeai`, used a different pattern with `genai.configure(...)` and `genai.GenerativeModel(...)`, that package is now deprecated. If you see tutorials online using that older pattern, it will likely still run for now, but Google recommends migrating away from it, so we're using the current SDK from the start.)
- `client.models.generate_content(model=..., contents=...)` sends the request to Google's servers and waits for a response, this is the correct current method name. (You may also see a similar-looking method called `client.interactions.create(...)` in Google's docs, that one is for a different feature, longer-running agent tasks, not simple one-off text generation, so it's not what we want here.)
- `response.text` extracts just the generated text from the full response object.

Checkpoint: What would happen if you ran this cell without completing Step 7 (the `.env` file)?

Answer: `os.getenv("GEMINI_API_KEY")` would return `None`, and `genai.Client(api_key=None)` would fail (or the following API call would fail) with an authentication error, since Google's servers have no way to identify which account is making the request. This is exactly why we verify the key loads correctly *before* trying to make a real API call, so the error is easy to diagnose instead of buried in a confusing network error message.

If you want stronger results: Google's Gemini lineup includes multiple models at different capability/cost tradeoffs. You can swap `"gemini-3.7-flash"` for a more capable model (e.g. a Pro-tier model) by changing that one string, the rest of your code stays exactly the same. This is one of the practical benefits of working through a hosted API instead of a locally-downloaded model: switching models never requires re-downloading anything or worrying about whether your hardware can handle it.

A note on model names specifically: Gemini model names change as Google releases new versions. `gemini-3.7-flash` is current as of this writing, but by the time you're reading this, a newer version may exist. If this exact model name gives you a "not found" error, check Google AI Studio's model list for the current name and swap it in, the rest of the code doesn't change.

However, to build a more useful LLM app, we often need it to work with specific knowledge or data. That's where the next part comes in: LLM apps with custom data via **Retrieval-Augmented Generation.**

2.7 Building a Simple Chatbot with LangChain (Using a Custom Dataset)

One powerful pattern for LLM applications is **Retrieval-Augmented Generation (RAG)**. The idea is:

1. Provide context to the LLM from an external knowledge source (our custom data).
2. Have the LLM generate answers using both its built-in knowledge and the provided context.

This helps overcome a limitation: LLMs like Gemini have a lot of knowledge up to their training cutoff, but they might not know specifics about your data (for example, a company policy document or, in our case, a student handbook). By giving the model relevant excerpts, we focus its attention and get more accurate, specific answers.

We will create a chatbot that can answer questions about a company information sheet. The dataset used in this walkthrough is `jollibee-info.pdf`, a document containing background information about the company, such as its history, menu, and mission. Our goal: allow a user to ask a question about the company, and the AI will answer based on that document.

Steps to build this chatbot:

1. Load and preprocess the dataset (the PDF text).
2. Split the text into manageable chunks and create embeddings (numerical representations) for each chunk.
3. Store these embeddings in a vector database for quick similarity search.
4. When a question is asked, convert the question to an embedding, find the most similar handbook chunk(s) from the vector store.
5. Feed those chunks along with the question to an LLM to generate an answer.
6. Return the answer.

We will use LangChain to simplify a lot of these steps, connected to Gemini via `langchain-google-genai`.

2.7.1 Data Ingestion and Splitting

First, let's load the PDF and split it into chunks of text. (A quick kernel sanity check runs in the next cell first, confirming VS Code is using the Python interpreter inside your virtual environment, not your system's default Python.)

In [22]:
```
import sys

print(sys.executable)
```

Out [22]:
```
c:\Users\Kein Jake\Downloads\DS314 Trial\venv\Scripts\python.exe
```

In [23]:
```python
# Requires: pip install langchain-community pypdf

from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

# 1. Load the PDF document
loader = PyPDFLoader("jollibee-info.pdf")
documents = loader.load()  # Reads the PDF and outputs a Document list (each page as a doc, typically)

print(f"Loaded {len(documents)} documents from the PDF.")
print(f"Sample document content (first 200 chars):\n{documents[0].page_content[:200]}")
```

Out [23]:
```
Loaded 6 documents from the PDF.
Sample document content (first 200 chars):
JOLLIBEE: FROM ICE CREAM PARLOR TO
 GLOBAL RESTAURANT GROUP
A consolidated research dossier on the origin, food, mascots, business strategy, growth, setbacks, partnerships,
acquisitions, achievements,
```

In the code above, `PyPDFLoader` is a LangChain utility that reads PDFs, imported from `langchain_community` (older tutorials sometimes import document loaders directly from `langchain`, that path is deprecated, `langchain_community` is correct). We then loaded all pages into `documents`. We print how many and a snippet to verify it worked. Typically, each page of the PDF becomes an element in `documents`.

However, each page might still be too large or contain multiple topics. Also, questions might be answered from a specific paragraph. So we will split the text further into chunks (e.g., a few hundred words each) using `RecursiveCharacterTextSplitter`, imported from the dedicated `langchain_text_splitters` package (again, some older tutorials import this from `langchain.text_splitter` directly, which still works but is the older path). This splitter will break text into chunks of a specified size, trying to do so at logical boundaries (like paragraph or sentence ends when possible).

In [24]:
```python
# 2. Split documents into smaller chunks for embedding
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=100
)

docs = text_splitter.split_documents(documents)

print(f"Split into {len(docs)} chunks.")
print("Example chunk:", docs[0].page_content[:100])
```

Out [24]:
```
Split into 22 chunks.
Example chunk: JOLLIBEE: FROM ICE CREAM PARLOR TO
 GLOBAL RESTAURANT GROUP
A consolidated research dossier on the o
```

Here we chose `chunk_size` of 1000 characters (roughly ~200 words) with a 100-character overlap between chunks (to avoid cutting important info). After splitting, we might have, say, a few hundred chunks (depending on PDF length).

2.7.2 Embeddings and Vector Store

Next, we convert each chunk of text into an embedding (a vector). We'll use **Google's embedding model**, accessed through `langchain-google-genai`'s `GoogleGenerativeAIEmbeddings` wrapper, since our official course model is Gemini and its embedding model is designed to pair with it directly.

We then store the embeddings in a vector database. We'll use **Chroma**, our official vector store, a simple, well-documented option that requires no separate server setup.

In [25]:
```python
# 3. Create embeddings and build the Chroma vector store

from langchain_google_genai import GoogleGenerativeAIEmbeddings
from langchain_chroma import Chroma
from dotenv import load_dotenv
import os

load_dotenv()

# Create an embedding model using Google's Gemini embedding model
embedding_model = GoogleGenerativeAIEmbeddings(
    model="gemini-embedding-2",
    google_api_key=os.getenv("GEMINI_API_KEY")
)

# Convert the text chunks into embeddings and store them in Chroma
vector_store = Chroma.from_documents(
    docs,
    embedding=embedding_model
)

print(
    f"Vector store built with {vector_store._collection.count()} embedded documents."
)
```

Out [25]:
```
Vector store built with 63 embedded documents.
```

This code will call Google's embedding API for each chunk of text, so it requires internet access and your API key. Once done, `vector_store` holds our data in a form that can be queried by similarity.

(Note: `Chroma` is imported from the dedicated `langchain_chroma` package here, the current recommended path. Older tutorials sometimes import it from `langchain_community.vectorstores` instead, which still works but is being phased out in favor of the dedicated package.)

(Note: If this is run repeatedly, you might want to persist the Chroma index to avoid re-computation. In LangChain, you can specify a `persist_directory` for Chroma. For simplicity, we did it in-memory here.)

Checkpoint: Why do we need to embed the *chunks* (from Section 2.7.1) rather than embedding the entire handbook as one single vector?

Answer: A single embedding for the whole document would blend together everything in it, company history, menu details, mission statement, all mashed into one vector, making it impossible to retrieve just the *relevant* part for a specific question. By embedding each chunk separately, we can find the one or two chunks that are actually closest in meaning to the user's question, and only feed those to the model, this is the entire mechanism that makes retrieval work.

2.7.3 Choosing an LLM for Q&A

Now we need an LLM that will generate answers. We'll use **Gemini**, wrapped in LangChain's `ChatGoogleGenerativeAI` class so it can plug directly into a LangChain chain, the same way the pipeline components we've used so far all connect together.

In [26]:
```
# Requires: pip install langchain-google-genai

from langchain_google_genai import ChatGoogleGenerativeAI
from dotenv import load_dotenv
import os

load_dotenv()

# 4. Load Gemini as the LLM for our QA chain
llm = ChatGoogleGenerativeAI(
    model="gemini-3.6-flash",
    google_api_key=os.getenv("GEMINI_API_KEY"),
    temperature=0.2  # lower temperature = more focused, less creative answers, good for factual Q&A
)

print("Gemini LLM initialized successfully.")
```

Out [26]:
```
Gemini LLM initialized successfully.
```

What we did:

- Created a `ChatGoogleGenerativeAI` instance pointed at `gemini-3.7-flash`, our official course model (matching the model used in the Section 2.6 demo, so the whole notebook is consistent).
- Set a low `temperature`, since we want factual, grounded answers about the company info sheet, not creative variation.
- LangChain can now use this `llm` object interchangeably with any other LLM wrapper in a chain, that interchangeability is one of the main reasons LangChain is used here.

2.7.4 Creating a QA Chain

LangChain makes it easy to create a QA system by combining the vector store and the LLM. We'll build this in two pieces, then connect them:

1. A **document chain** (`create_stuff_documents_chain`) that takes retrieved chunks and "stuffs" them into a prompt template, then sends that to the LLM.
2. A **retrieval chain** (`create_retrieval_chain`) that connects a retriever (which finds the relevant chunks) to that document chain, so the whole thing runs in one step: question in, grounded answer out.

Let's set that up:

In [27]:

```python
# 5. Create a retriever and a RAG question-answering chain

from langchain_core.prompts import ChatPromptTemplate
from langchain_classic.chains import create_retrieval_chain
from langchain_classic.chains.combine_documents import create_stuff_documents_chain

# Create a retriever that returns the 2 most relevant chunks
retriever = vector_store.as_retriever(
    search_kwargs={"k": 2}
)

# Create a prompt that tells Gemini to answer using the retrieved context
prompt = ChatPromptTemplate.from_template(
    """Answer the question based only on the following context.

<context>
{context}
</context>

Question: {input}

If the answer cannot be found in the context, say that the information is not available in the provided document."""
)

# Combine the retrieved documents with the LLM
document_chain = create_stuff_documents_chain(
    llm,
    prompt
)

# Connect retrieval and generation
qa_chain = create_retrieval_chain(
    retriever,
    document_chain
)

print("RAG Q&A chain created successfully.")
```

Out [27]:

```
RAG Q&A chain created successfully.
```

A few things worth noticing in the cell above:

- `retriever = vector_store.as_retriever(search_kwargs={"k": 2})` sets up retrieval of the top 2 most similar chunks for each query. You can adjust `k`, more might give the model more context, but too many can introduce irrelevant info or exceed input limits.
- The **prompt template** explicitly instructs the model to answer only from the provided `{context}`, and to say the information isn't available if it can't be found there. This one instruction is doing real work, it's what should stop the model from confidently guessing when the document doesn't actually contain the answer.
- `create_stuff_documents_chain(llm, prompt)` builds the piece that inserts retrieved chunks into that prompt and sends it to the model.
- `create_retrieval_chain(retriever, document_chain)` connects retrieval and generation into one runnable chain.

(Note: this `create_retrieval_chain` / `create_stuff_documents_chain` pattern is LangChain's current, recommended way to build a RAG chain. An older class called `RetrievalQA` does something similar with less code, and still works, but is considered a simpler, older-style pattern. We're using the current pattern here since it also lets us customize the prompt directly, which is what makes the "say you don't know" instruction above possible.)

2.7.5 Testing the Chatbot

Now, everything is set. Let's ask a sample question to our QA chain. For example, a question answerable from the Jollibee info sheet:

In [30]:
```
# 6. Ask a question using the RAG chain

question = "Enumerate all the products Jollibee offers."

result = qa_chain.invoke({
    "input": question
})

print("Q:", question)
print("A:", result["answer"])
```

Out [30]:
```
c:\Users\Kein Jake\Downloads\DS314 Trial\venv\Lib\site-packages\langchain_google_genai\chat_models.py:3719: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature will be ignored.
  request = self._build_request_config(
Q: Enumerate all the products Jollibee offers.
A: Based on the provided context, the products offered include:

* Chickenjoy (and fried chicken)
* Yumburger
* Sweet-style Jolly Spaghetti
* Palabok
* Burgers
* Rice meals (including traditional Filipino breakfast rice meals)
* Desserts
* Coffee
* Tea
* Chinese cuisine
* Pizza
* Baked goods
* Barbecue
* Hot pot
* Hot mais con yelo (sweet corn) served with milk and crushed ice
* Tuna Pie
* Pies-to-Go
```

If our system is working, the answer should reflect information from the Jollibee info sheet, not the model's general background knowledge about the company from its training data. The model, given the relevant chunk from the document, should output a sentence or two answering the question, grounded in that specific document.

Another example:

In [31]:
```
question = "According to the document, what is Jollibee's mission?"

result = qa_chain.invoke({
    "input": question
})

print("Q:", question)
print("A:", result["answer"])
```

Out [31]:
```
c:\Users\Kein Jake\Downloads\DS314 Trial\venv\Lib\site-packages\langchain_google_genai\chat_models.py:3719: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature will be ignored.
  request = self._build_request_config(
Q: According to the document, what is Jollibee's mission?
A: Based on the provided context, the information about Jollibee's specific mission is not available in the document (it mentions that Exhibit 3 reproduces the company's values, vision, and mission, but Exhibit 3 is not included in the text).
```

This should retrieve the section of the document that actually discusses the company's mission, and the model will phrase an answer based on that content, not a generic answer pulled from what it already knew about Jollibee before we gave it this document.

We can treat `qa_chain.invoke(...)` as our chatbot's brain: it takes any user question about the company info sheet and returns a grounded answer. In an interactive app, you would loop this, or perhaps integrate with a chat interface. Here, we're just demonstrating via prints.

Note: The quality of answers will depend on retrieval quality and prompt phrasing, not just the model. Gemini generally handles this style of task well, and our prompt template (Section 2.7.4) explicitly tells it to say the information isn't available if it can't be found in the retrieved context, but that instruction is a safeguard, not a guarantee. Always be cautious: these models do not inherently know when to say "I don't know", they can still try to answer anyway if the instruction is weak or the retrieved context is misleading. In a refined application (something we'll build toward later in the course), you would add additional checks, like requiring a minimum similarity score before trusting a retrieved chunk at all.

2.7.6 End-to-End Example Interaction

Let's simulate a short conversation with our Jollibee info chatbot to see it in action:

In [32]:

```
questions = [
    "When and where was Jollibee founded?",
    "What are some of Jollibee's well-known menu items?",
    "Does Jollibee operate internationally, and if so, where?",
]

for q in questions:
    result = qa_chain.invoke({
        "input": q
    })

    print(f"Q: {q}\nA: {result['answer']}\n")
```

Out [32]:

```
c:\Users\Kein Jake\Downloads\DS314 Trial\venv\Lib\site-packages\langchain_google_genai\chat_models.py:3719: UserWarning: Model 'gemini-3.6-
flash' uses fixed sampling defaults; the sampling parameter(s) temperature will be ignored.
  request = self._build_request_config(
Q: When and where was Jollibee founded?
A: Based on the provided context, the family's original business began in **1975 in Manila** as an ice-cream business, and **Jollibee Foods
Corporation was born in 1978** when it fully pivoted into its fast-food identity.

c:\Users\Kein Jake\Downloads\DS314 Trial\venv\Lib\site-packages\langchain_google_genai\chat_models.py:3719: UserWarning: Model 'gemini-3.6-
flash' uses fixed sampling defaults; the sampling parameter(s) temperature will be ignored.
  request = self._build_request_config(
Q: What are some of Jollibee's well-known menu items?
A: Based on the provided context, some of Jollibee's well-known and favorite menu items include:

* Chickenjoy
* Yumburgers (and other burger choices like mushroom, garlic, and cheese)
* Spaghetti Special / sweet-style Jolly Spaghetti
* Jolly Hotdog
* French fries
* Palabok
* Rice meals (such as Honey Beef Rice and Shanghai Rolls)
* Desserts (such as Ice Craze in Buko pandan and mais con yelo)

c:\Users\Kein Jake\Downloads\DS314 Trial\venv\Lib\site-packages\langchain_google_genai\chat_models.py:3719: UserWarning: Model 'gemini-3.6-
flash' uses fixed sampling defaults; the sampling parameter(s) temperature will be ignored.
  request = self._build_request_config(
Q: Does Jollibee operate internationally, and if so, where?
A: Yes, Jollibee operates internationally. Based on the provided context, it has operated in the following international locations:

* Two islands in the NW Pacific
* Vietnam
* Indonesia
* Dubai
* Kuwait
* The United States (US)
* Several other Asian and Middle Eastern locations
```

Each question:

- The first should find founding details, if the document includes company history.
- The second should retrieve a list of menu items mentioned in the document.
- The third should find whatever the document says about international operations, if anything, if the document doesn't mention this at all, the answer should correctly say the information isn't available, rather than guessing based on general knowledge. This is a good test of whether our retrieval and prompt instructions are actually working together correctly.

By examining the answers, we can evaluate if our LLM app is correctly using the handbook data. Ideally, it should ground its phrasing in the handbook's actual content.

Congratulations, we've essentially built a domain-specific Q&A chatbot, one that answers from a specific document instead of just general training knowledge! What we did here is a blueprint for many real-world LLM apps:

- Use embeddings to give the model knowledge it didn't originally have (our custom data).
- Use an LLM to generate a grounded answer based on that knowledge.

2.7.7 Troubleshooting Common VS Code Setup Errors

Since this is likely your first time running a real LLM pipeline locally in VS Code, here are the most common errors you might hit, and what they actually mean:

Error	Likely Cause	Fix
`ModuleNotFoundError: No module named 'google.generativeai'`	Your virtual environment isn't activated, or VS Code's selected kernel isn't pointing at your `venv`	Re-check Step 4 (activation) and Step 6 (kernel selection) above
`AttributeError` involving `genai.configure` or `genai.GenerativeModel`	You copy-pasted code from an older tutorial using the deprecated `google-generativeai` package instead of the current `google-genai` package this notebook uses	Use `from google import genai` and `client = genai.Client(...)`, not `import google.generativeai as genai`
`FileNotFoundError: jollibee-info.pdf`	The PDF isn't in the same folder as this notebook, or the filename doesn't match exactly	Confirm the file is in your project folder and the filename matches exactly, including spaces and capitalization
`google.api_core.exceptions.Unauthenticated` or similar auth error	Your API key wasn't loaded correctly	Re-run the Step 8 verification cell first; if that fails, re-check your `.env` file's name and contents
Notebook runs very slowly on the embeddings step	Normal, each chunk requires a separate API call to Google's embedding model, larger handbooks mean more calls	Be patient on first run; consider reducing chunk count for testing by slicing `docs[:20]` temporarily
`RateLimitError` or `429` error	Too many requests sent too quickly on the free tier	Wait a minute and re-run; if it persists, reduce how often you re-run cells while testing

2.8 Reflection on Tools and Techniques

Let's reflect on what tools we used and why:

- **LangChain:** simplified integrating all components (loader, text splitter, embeddings, vector store, LLM, chain) into one consistent interface. Without LangChain, we could still do this by manually coding each part, LangChain just provides a higher-level abstraction and common interface.
- **Embeddings + Vector Store:** this is crucial for handling large documents. LLMs have input size limits, we can't feed a whole handbook every time. Instead, we index the knowledge and retrieve relevant bits on demand. This technique enables "chat-with-your-data" type applications.
- **Gemini as our model:** we chose Gemini specifically because it's free-tier accessible and requires no local hardware, letting every student in this course run the exact same pipeline without cost or GPU barriers. The beauty of this setup is that we could change one line (the model name) and the rest of the pipeline stays the same, that's the benefit of building on LangChain's standard interfaces.
- **RetrievalQA chain:** abstracted away prompt handling. If building from scratch, you'd have to craft this prompt manually. LangChain's chain does it for us and runs the model.

Finally, always remember to evaluate and test. We should verify if the answers truly align with the source. If not, we might need to adjust chunk sizes, number of retrieved docs, or the prompt. When deploying such a system, consider adding a citation or reference (so users trust that the answer came from an authoritative source), or a disclaimer that it's based on the 2023 handbook and might not reflect later changes.

We have now covered the journey from understanding LLM concepts to building a functioning LLM application. Next, it's time for you to practice these concepts in a lab setting.

Laboratory Task 2

Buddy System — To be completed and submitted in pairs

This week, there is no separate written assignment, **this lab is the deliverable.**

Your task is to recreate the entire pipeline from Section 2.7 above, end to end, using your own document instead of `jollibee-info.pdf`, and then test the chatbot using questions from both partners.

Names: *(fill in both partners)* 1. 2.

Your chosen document: *(fill in the filename you used)*

Part 1: Recreate the Pipeline (with your own PDF)

Working with your buddy:

1. Choose **one PDF document**, anything text-heavy works: a resume, a class syllabus, a company brochure, a school handbook excerpt, a product manual. Place it in your project folder.
2. Rebuild each step from Section 2.7, adapted to your file:
   - Load your PDF with `PyPDFLoader`, using your actual filename
   - Split it into chunks with `RecursiveCharacterTextSplitter`
   - Embed the chunks and build a Chroma vector store
   - Set up `ChatGoogleGenerativeAI` as your LLM
   - Build your retrieval chain (`create_retrieval_chain` + `create_stuff_documents_chain`), keeping the same prompt instruction that tells the model to say when information isn't available in the context

> *Since this is a pair activity, you only need to build **one shared chatbot pipeline**.*

Part 2: Testing the Chatbot

Each partner must individually test the shared chatbot.

3. Ask a total of **10 questions** about your chosen document:
   - **Partner 1:** 5 questions
   - **Partner 2:** 5 questions
4. For each question, record:
   - The question
   - The model's answer
   - The actual answer based on the document

Part 3: Trap Question

Among your questions, include **one deliberate "trap" question**: something a person might reasonably ask about the topic of your document, but that your specific document does **not** actually contain the answer to.

Run it through your chain exactly like the other questions and record the result.

Assess whether the model correctly said the information wasn't available or hallucinated a plausible-sounding but made-up answer.

If it hallucinated, try adjusting your prompt template's wording (Section 2.7.4) to make the "say you don't know" instruction stronger, and re-run the trap question. Report whether that changed the result.

Part 4: Individual Reflections

Each partner must write their **own reflection based on their individual set of five questions**.

This means your notebook should contain **two separate reflections**, one from each partner.

Each reflection should be **150–250 words** and discuss what their own questions revealed about the reliability, grounding, and limitations of the RAG chatbot.

What to Submit

One shared notebook per pair, containing:

- Both names and the filename of the document you used
- Your rebuilt chatbot pipeline (all cells, actually run, with real output visible, not just code)
- **10 questions and answers in total**
  - 5 questions from Partner 1
  - 5 questions from Partner 2

- The actual answers based on the document
- Your trap question, its answer, and your grounded/hallucinated assessment
- Prompt adjustment and re-test, if hallucination occurred
- **Two separate 150–250 word reflections**, with each reflection based on that student's own five questions

Due: Before Week 3 session begins.

Grading Rubric

Criterion	Points	What's being checked
Pipeline runs end-to-end on your own document	25	Loader, splitter, embeddings, vector store, and chain all work without errors, using your own filename and content
Question testing and answer validation	25	10 questions in total, with 5 contributed by each partner; answers are checked against the document's actual content
Trap question handled and honestly assessed	20	The trap question was genuinely unanswerable from the document, and the pair correctly identified whether the model hallucinated or correctly declined to answer
Attempted fix (if hallucination occurred)	10	If the model hallucinated, the pair attempted a prompt adjustment and reported the result; if it didn't hallucinate, full credit is given automatically
Individual reflections	20	Two separate reflections demonstrate each partner's understanding based on their own five questions
Total	**100**	